

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 4

SCS 1101, July 2005 Supplementary

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Supplementary paper · answer any 5 of 6 · 20 marks each

Book 4 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide **How to use this book**

Glance **At a glance**

Q1 **Question 1 [20]**

- ▸ (i) Describe RAM and ROM and their function [4]
- ▸ (ii) Types of RAM and ROM [12]
- ▸ (iii) Applications [4]

Q2 **Question 2 [20]**

- ▸ Major milestones in the history and evolution of computers

Q3 **Question 3 [20]**

- ▸ (i) Purpose of a compiler and the tasks it performs [10]
- ▸ (ii) An example IDE and its main components [10]

Q4 **Question 4 [20]**

- ▸ Internet access through a LAN or an ISP: the options [20]

Q5 **Question 5 [20]**

- ▸ (i) Components of structured programming, with code snippets [10]
- ▸ (ii) Importance of declaring a variable [2]
- ▸ (iii) Three basic data types in C [6]
- ▸ (iv) What is an abstract data type? [2]

Q6 **Question 6 [20]**

- ▸ (i) The concept of a string in C, with an example [6]
- ▸ (ii) The three components of a function header [6]
- ▸ (iii) Function: Fahrenheit to Celsius, C = 5/9 (F − 32) [8]

Refs **Sources for this book**

Links **Source register**

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, July 2005 Supplementary** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

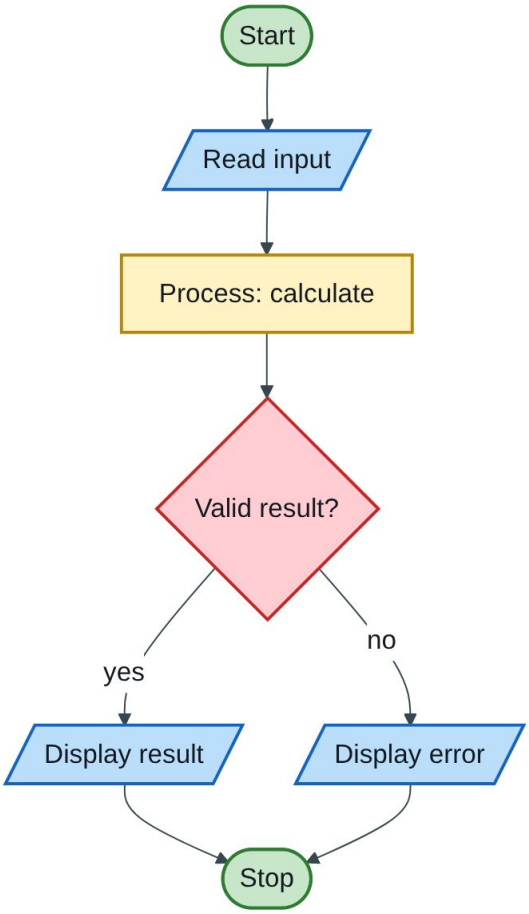


Figure 4.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

3 hours, answer **any five of six**, 20 marks each. This paper is mostly theory, so the marks go to complete, well-organised lists. Drop either Q2 (history essay) or Q4 (internet access), whichever you know less well.

Question	What it tests	Pick?
Q1	RAM and ROM: description, types, applications	Yes
Q2	Milestones in computer history	Yes, if you learn the timeline
Q3	Compiler purpose and tasks; IDE components	Yes
Q4	Internet access through a LAN or an ISP	Optional
Q5	Structured programming, declarations, data types, ADT	Yes
Q6	Strings, function header, Fahrenheit → Celsius	Yes

Question 1 [20]

Like you're 3: RAM is your desk: you spread out today's work, and the cleaner wipes it every night (switch off = gone). ROM is a printed book on the shelf: you can read it any time, but you cannot write in it, and it is still there tomorrow.

(i) Describe RAM and ROM and their function [4]

- **RAM (Random Access Memory)** — read/write main memory that holds the operating system, programs and data **currently in use**; it is fast and **volatile**, so its contents are lost when power is off [2].
- **ROM (Read Only Memory)** — memory whose contents are written during manufacture (or programming) and normally only read; it is **non-volatile** and stores permanent start-up instructions such as the BIOS/firmware that boots the computer [2].

(ii) Types of RAM and ROM [12]

Name [1] and describe [1] each type:

Type	Family	Description
SRAM (Static RAM)	RAM	Stores each bit in a flip-flop; very fast and needs no refreshing, but costly and bulky; used for CPU cache.
DRAM (Dynamic RAM)	RAM	Stores each bit as charge on a tiny capacitor that must be refreshed thousands of times a second; cheaper and denser; used for main memory (SDRAM, DDR4, DDR5).
Mask ROM	ROM	Data is built into the chip during manufacture and can never be changed; cheapest in large volumes.
PROM (Programmable ROM)	ROM	Supplied blank and programmed **once** by the user with a PROM programmer (fuses are blown).
EPROM (Erasable PROM)	ROM	Can be erased by exposing the chip's quartz window to **ultraviolet light**, then reprogrammed.
EEPROM / Flash	ROM	Erased and rewritten **electrically**, in place, without removing the chip; flash memory is a fast block-erasable EEPROM used in USB disks, SSDs and BIOS chips.

(iii) Applications [4]

- RAM: main memory for running programs and open files; SRAM as CPU cache [2].

- ROM: BIOS/UEFI start-up firmware in PCs; firmware in embedded devices such as microwaves, printers and calculators; flash in USB drives and phones [2].

Engineering lens: the **PLC** (programmable logic controller) in a pump station keeps its control program in flash/EEPROM so it survives power cuts and restarts the pumps automatically, while its working values (current levels, timers) live in RAM.

Examiner's tip: for 12 marks, give six types with one clear sentence each. Two RAM types plus four ROM types is the standard answer.

Question 2 [20]

Major milestones in the history and evolution of computers

Like you're 3: people first counted with beads, then with gear wheels, then with glowing glass tubes, then with tiny switches on a chip, and now with phones that talk to the whole world.

Aim for ten milestones, 2 marks each: date and name [1], why it mattered [1]. Write them in date order.

Date	Milestone	Why it mattered
Ancient times	**Abacus**	First counting aid using beads on rods.
1642	**Pascal's calculator (Pascaline)**	First widely known mechanical adding machine, using gears.
1801	**Jacquard loom**	Punched cards controlled a weaving pattern: the idea of a stored, changeable program.
1830s	**Babbage's Analytical Engine**; Ada Lovelace's notes (1843)	Design for a general-purpose machine with a "mill" (CPU) and "store" (memory); Lovelace wrote the first published algorithm for it.
1890	**Hollerith's tabulating machine**	Punched cards processed the US census far faster; his company later became IBM.
1936	**Turing machine**	Alan Turing's theory of what any computer can calculate.
1946	**ENIAC**	Early general-purpose electronic digital computer, built from vacuum tubes (first generation).
1945–1953	**MIT Whirlwind**	First real-time high-speed computer with random-access **magnetic-core memory**, which became the standard main memory for decades (MIT-3).
1947	**Transistor** (Bell Labs)	Replaced vacuum tubes: smaller, cooler, more reliable (second generation).
1958–59	**Integrated circuit**	Many transistors on one chip (third generation).
1969	**ARPANET**	The first message of the network that became the internet was sent from UCLA's Boelter Hall on 29 October 1969 (UCLA-1); UCLA was the first node (UCLA-2).
1971	**Microprocessor** (Intel 4004)	A whole CPU on one chip (fourth generation).

Date	Milestone	Why it mattered
1981	**IBM PC**	Personal computers enter offices and homes.
1989–91	**World Wide Web** (Tim Berners-Lee, CERN)	Easy, linked access to information over the internet.
2000s–today	**Mobile, cloud and AI**	Smartphones, cloud computing and artificial intelligence (the fifth generation).

Engineering lens: civil engineers were early computer users. MIT's civil engineering department developed **COGO** (coordinate geometry) in the early 1960s, one of the first programming systems written for surveyors and highway designers. Today the same line leads to CAD, BIM and hydraulic modelling software.

Examiner's tip: dates within a few years are fine; what scores is the **order** and the **reason each step mattered** (smaller, faster, cheaper, more reliable).

Question 3 [20]

(i) Purpose of a compiler and the tasks it performs [10]

Like you're 3: you write a story in English; the computer only reads 0s and 1s. The compiler is the translator who reads your whole story, checks the spelling and grammar, and hands back the robot-language version in one go.

Purpose [2]: a compiler translates a whole program written in a high-level language (the **source code**, e.g. `prog.c`) into machine code (the **object code**) that the CPU can execute, and reports errors found during translation.

Tasks, in order (the phases of compilation) [8]:

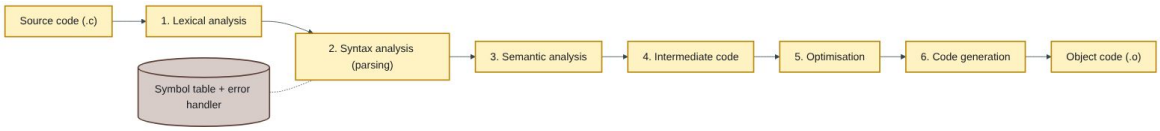


Figure 4.2 — Flowchart

1. **Lexical analysis** — breaks the text into tokens (keywords, names, numbers, symbols) and removes comments.
2. **Syntax analysis (parsing)** — checks the tokens follow the grammar of C; reports syntax errors such as a missing `;`.
3. **Semantic analysis** — checks meaning: variables declared, types compatible.
4. **Intermediate code generation** — produces a simple, machine-independent form of the program.
5. **Optimisation** — improves the code to run faster or use less memory.
6. **Code generation** — produces the final machine code for the target CPU.
7. Throughout: keeps a **symbol table** of all names, and **reports errors** with line numbers.

The **linker** then joins the object code with library code (e.g. `printf`) to make the executable. Compilers are not all identical: UCLA's introductory programming course requires every submitted program to run correctly under two different compilers, which trains students to write portable code (UCLA-3).

(ii) An example IDE and its main components [10]

Like you're 3: an IDE is a workshop where every tool you need hangs on one wall: notebook, translator, glue and magnifying glass.

Definition and example [2]: an Integrated Development Environment brings everything needed to write, build and test programs into one application. Examples for C: **Code::Blocks**, Dev-C++, Visual Studio, VS Code.

Main components of Code::Blocks [8, 2 each for any four]:

1. **Source code editor** — typing code with syntax colouring, auto-indent and auto-complete.
2. **Compiler and build tools** — GCC, run by a Build button, turns code into an executable; build messages appear in a log window.
3. **Linker** — joins your object files with libraries.
4. **Debugger** (GDB) — breakpoints, step through line by line, watch variable values.
5. **Project manager** — organises the files of a multi-file program.
6. **Run console / output window** — shows the program's input and output.

Examiner's tip: name a real IDE, then describe **four** components clearly; a list of names without explanation scores about half.

Question 4 [20]

Internet access through a LAN or an ISP: the options [20]

Like you're 3: to reach the big world of the internet, you either walk through your school's front gate (a LAN shares one connection), or your own house gets its own road from a road company (an ISP).

How the two fit together:

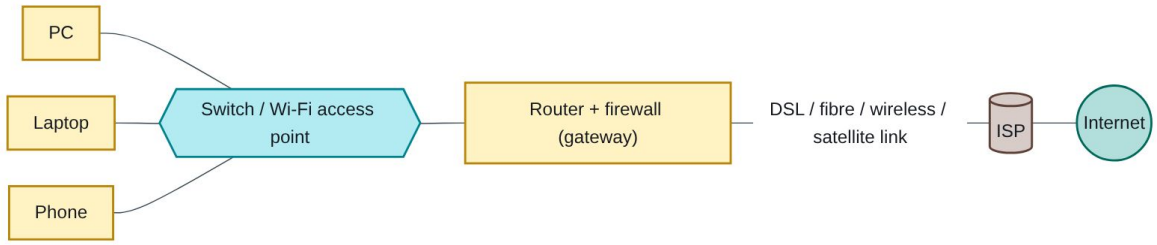


Figure 4.3 — Flowchart

A. Access through a LAN [8] — an organisation's computers share one internet connection through a gateway.

1. **Wired Ethernet LAN** — PCs plug into switches; a router/gateway connects the LAN to the ISP. Fast and reliable for offices and labs.
2. **Wireless LAN (Wi-Fi)** — access points let laptops and phones join without cables; convenient, but shared and less secure unless encrypted.
3. **Proxy server** — one server fetches web pages for all users, caching popular pages and filtering content.
4. **Leased line / dedicated link** — the organisation rents a permanent, guaranteed-speed link (often fibre) from a provider for the whole LAN.

B. Access through an ISP [12] — a home or business subscribes directly. Name [1] and describe [1] six options:

Option	How it works	Typical features
Dial-up	Modem uses an ordinary telephone line, dialling the ISP for each session	Very slow; phone line busy while online; now obsolete
DSL / ADSL	Digital signal over the existing copper telephone line, without blocking calls	Always on; speed falls with distance from the exchange
Fibre (FTTH / FTTB)	Light pulses through optical fibre to the home or building	Fastest and most stable; limited to areas with fibre
Cable broadband	Through the cable-TV coaxial network	Fast; bandwidth shared by neighbours
Mobile broadband (3G/4G/5G)	SIM card in a phone, dongle or router connects through cellular towers	Available almost anywhere with coverage; data-bundle costs
Fixed wireless / WiMAX	Radio link from a rooftop antenna to a provider's tower	Good where cables do not reach
Satellite (VSAT or low-earth-orbit)	Dish communicates with a satellite	Works in remote rural areas; higher cost; older systems have high latency

Engineering lens: a construction site office far from town might use **mobile broadband or satellite**, while a city engineering firm uses **fibre** to the office LAN; remote borehole telemetry often uses the **mobile network**.

Examiner's tip: this is a "brief notes" question: use two clear headings (LAN, ISP) and keep each option to one line of how it works plus one line of pros and cons.

Question 5 [20]

(i) Components of structured programming, with code snippets [10]

Like you're 3: every program, however big, is built from only three Lego shapes: do things **in order**, **choose** between paths, and **repeat**. Then we put groups of blocks into labelled boxes (functions).

Structured programming builds programs only from these control structures, with no jumping around (`goto`), so programs are easier to read, test and maintain [2].

1. Sequence [2] — statements run one after another, top to bottom.

```
scanf("%lf", &d);
area = 3.14159 * d * d / 4.0;
printf("Area = %.4f\n", area);
```

2. Selection [2] — a condition chooses which statements run (`if`, `if-else`, `switch`).

```
if (area > 0.05)
    printf("Large pipe\n");
else
    printf("Small pipe\n");
```

3. Iteration (repetition) [2] — statements repeat while a condition holds (`for`, `while`, `do-while`).

```
for (i = 1; i <= 5; i++)
    printf("Reading %d\n", i);
```

4. Modularity (functions) [2] — the program is split into small, named functions, each doing one job.

```
double pipeArea(double d) { return 3.14159 * d * d / 4.0; }
```

(ii) Importance of declaring a variable [2]

Declaring tells the compiler the variable's **name and type**, so it can reserve the right amount of memory and check that the variable is used correctly [1]. It also catches spelling mistakes: using an undeclared name is a compile error, not a silent bug [1].

(iii) Three basic data types in C [6]

Type	Holds [1]	Size and example [1]
`int`	Whole numbers, positive or negative	Usually 4 bytes; `int bags = 50;`
`float` / `double`	Numbers with decimals	4 / 8 bytes; `double flow = 0.125;`
`char`	A single character	1 byte; `char grade = 'A';`

(iv) What is an abstract data type? [2]

An **abstract data type (ADT)** is a data type defined by **what operations it allows**, not by how it is stored inside [1]. Example: a **stack** is defined by push, pop and peek; it can be built from an array or a linked list without the user knowing [1].

Like you're 3: a TV remote is an ADT. You know what the buttons do; you do not need to know the wiring inside.

Question 6 [20]

(i) The concept of a string in C, with an example [6]

Like you're 3: a string is a row of letter beads on a thread, with a special stopper bead at the end so you know where the word stops. In C the stopper is `'\0'`.

- C has no separate string type; a string is an **array of `char`** ending with the **null character `'\0'`** [2].
- The array must be one longer than the text: `"NUST"` needs 5 boxes: N, U, S, T, \0 [1].
- Strings are read with `scanf("%s", name)` (stops at a space) or `fgets` (reads spaces), printed with `%s`, and handled with `<string.h>` functions such as `strlen`, `strcpy`, `strcmp` and `strcat` [1].

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char city[20] = "Bulawayo";           /* stores B,u,l,a,w,a,y,o,\0 */
    printf("%s has %zu letters\n", city, strlen(city));   /* 8 */
    return 0;
}
```

Example [2].

(ii) The three components of a function header [6]

Like you're 3: a function header is the label on a machine: what comes out, the machine's name, and what you must put in.

```c
double   pipeArea   (double d)
/* (1)      (2)        (3)      */
```

1. **Return type** — the type of value the function sends back (`double`; `void` if nothing) [2].
2. **Function name** — the identifier used to call it (`pipeArea`) [2].
3. **Parameter list** — in brackets, the type and name of each input (`double d`); empty or `void` if none [2].

(iii) Function: Fahrenheit to Celsius, C = 5/9 (F − 32) [8]

Like you're 3: take the Fahrenheit number, subtract 32, then shrink it to five-ninths of its size.

```c
#include <stdio.h>

/* returns the Celsius equivalent of a Fahrenheit temperature */
double fahrenheitToCelsius(double f)
{
    return 5.0 / 9.0 * (f - 32.0);       /* 5.0/9.0, NOT 5/9 */
}

int main(void)                           /* shows the function working */
{
    double f;
    printf("Temperature in Fahrenheit: ");
```

```c
    scanf("%lf", &f);
    printf("%.1f F = %.1f C\n", f, fahrenheitToCelsius(f));
    return 0;
}
```

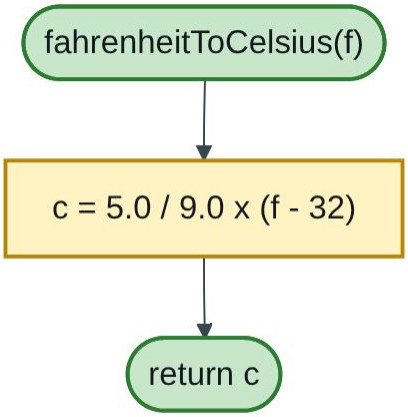


Figure 4.4 — Flowchart

Tests: 212 → 100.0; 32 → 0.0; 98.6 → 37.0; −40 → −40.0.

Marking guide [8]: header with `double` return type [2]; parameter [1]; formula written with real numbers [3]; `return` [1]; call shown from `main` [1].

The trap that costs most students 3 marks: in C, `5 / 9` is **integer division** and equals **0**, so `5/9 * (f - 32)` always returns 0. Writing `5.0 / 9.0` makes it real division.

Engineering lens: temperature matters in water engineering because water's **viscosity and density change with temperature**, which affects pipe friction losses and settling in treatment tanks; field instruments often report in one scale while design tables use the other.

Sources for this book

- [MIT-3] Whirlwind Computer, 1944–59 (IEEE Milestone) — magnetic-core memory milestone (Q2).
- [UCLA-1] UCLA Samueli — first ARPANET message, 1969 (Q2).
- [UCLA-2] Birthplace of the Internet (IEEE Milestone) — UCLA as the first node (Q2).
- [UCLA-3] UCLA CS 31 syllabus — programs must run under two compilers (Q3).
- [MIT-1] MIT 6.087 — C strings and functions (Q5, Q6).
- Textbook knowledge (tier 4): memory types, compiler phases, IDE components, internet access options, data types, ADTs.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	✓
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	Further reading
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	✓
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	✓
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	✓
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	✓
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading